前言

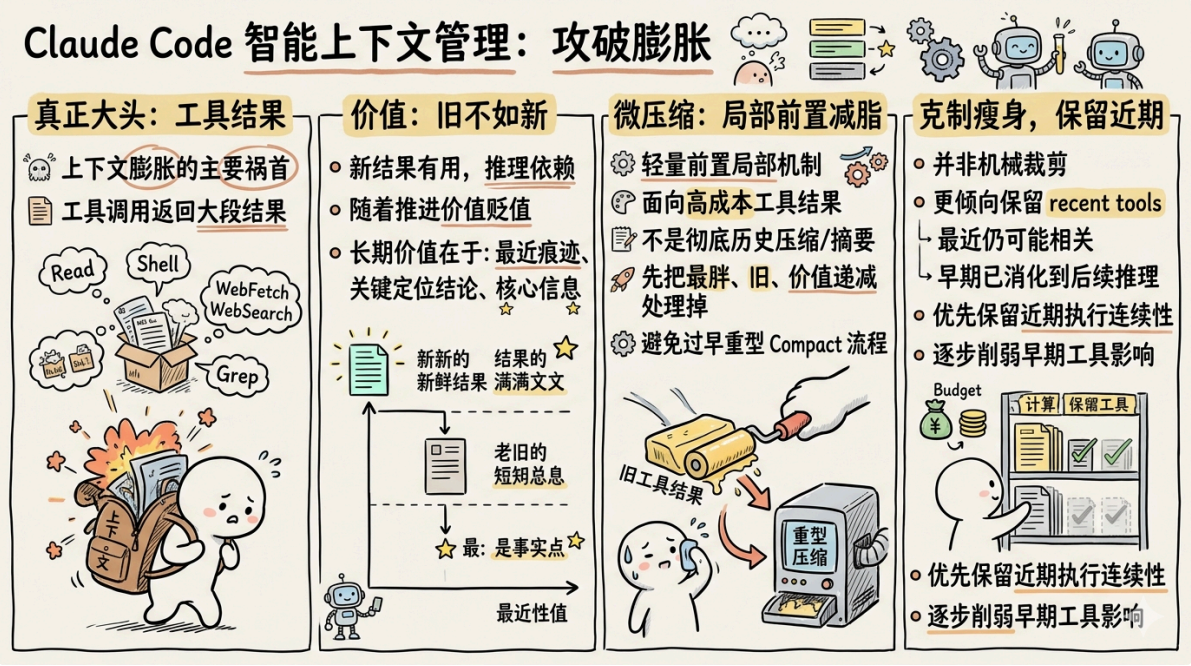
大家都知道模型的上下文是有限的，为了能够继续工作，现在往往有两种方案：一种是日志追踪（AGENTS.md、Harness），另一种是上下文压缩。

上下文压缩并不是单纯的把所有的`token`基于LLM做一次压缩。基于此，本章从`Claude Code`的上下文压缩技术进行分析。

`Claude Code` 是采用了分层处理的技术：

1. 先清理最占 token、但信息密度最低的旧工具结果
2. 如果长期记忆已经抽取出来，优先用结构化 session memory 代替整段旧历史
3. 再不够时，才走真正的摘要式压缩，把旧消息总结成一段“可继续工作的上下文”

本章主要讨论`Claude Code` 如何处理旧工具结果



一、microcompact

Claude Code 对上下文膨胀的判断其实很务实：
在长会话里，真正最容易把上下文撑爆的，往往不是用户新增的几句话，也不是 assistant 的普通回复，而是**工具调用返回的大段结果。**

尤其是下面这些工具，它们的输出通常天然偏长，而且很容易在多轮对话中不断累积：

- `Read`
- `Shell`
- `Grep`
- `Glob`
- `WebSearch`
- `WebFetch`
- `Edit`
- `Write`

这些工具的共同特点是：**单次返回很贵，但长期价值并不总是线性增长。**

例如：

- `Read` 可能返回整段文件内容
- `Grep` 和 `Glob` 可能返回大量匹配结果
- `WebFetch` 和 `WebSearch` 往往会带回大段网页文本
- `Shell` 可能输出长日志或命令结果

这些内容在刚产生时通常很有用，因为它们为模型提供了当前推理所依赖的原始材料；但随着会话继续推进，较早的一批工具结果往往会迅速贬值。真正长期有价值的，通常只是：

- 最近几次工具执行痕迹
- 关键定位结论
- 已经抽取出来的核心信息

而不是完整的原始输出全文。

这也是 Claude Code 选择优先处理 `tool result` 的原因：
它们往往既“最胖”，又最容易随着时间推移变得“不再值得继续完整保留”。

从这个角度看，`microcompact` 的定位并不是一次彻底的历史压缩，也不是直接把整段会话做成摘要。
它更像是一层**轻量、前置、面向高成本工具结果的局部减脂机制。**

它要做的事情可以概括成一句话：

> 先把最胖、最旧、后续价值递减最快的 `tool result` 处理掉，尽量让系统不要过早进入更重型的 compact 流程。

这里的“瘦身”也不是简单粗暴地把消息删光。
恰恰相反，Claude Code 在这一步处理得相当克制：它优先动那些体积大、衰减快的工具结果，而不会一上来就改写整段对话历史。

由于工具结果并不是“越多越好”，Claude Code 的思路不是机械地平均裁剪，而是更倾向于保留 **recent tools**，也就是最近仍然可能对当前推理有帮助的那部分工具调用痕迹。

这背后的直觉很好理解：

- 最近的 `tool result` 更可能仍与当前问题直接相关
- 更早的 `tool result` 即使曾经重要，也更可能已经被 assistant 消化进后续推理过程
- 如果全部原样保留，只会持续占据 token 预算

因此，`microcompact` 的核心并不是“保留所有工具历史”，而是：

> 优先保留近期执行连续性，逐步削弱更早期工具结果对上下文预算的占用。

不过，这种“瘦身”并不是单一策略完成的。 对于旧 `tool result`，Claude Code 并没有采用一种统一的直接删除策略，而是根据**当前是否还值得维护 prompt cache**，分成了两种不同处理方式：

- **Cached microcompact**
- **Time-based microcompact**

这两条路径表面上都在处理旧 `tool result`，但它们背后的出发点其实完全不同。

1. Cached microcompact

这一条路径的前提是：**服务端 prompt cache 仍然值得尽量维护。**

因此，它的重点不是直接把消息改短，而是尽量在**不破坏缓存前缀可复用性**的前提下，让旧工具结果从后续请求中逻辑上退出。
换句话说，它优先优化的是“缓存还能保住吗”。

2. Time-based microcompact

这一条路径的前提则相反：**缓存大概率已经冷掉，不值得继续为它付出约束。**

这时系统就不再执着于维持缓存命中，而是直接在请求前清理掉更早的 `tool_result.content`，换取一次更彻底的上下文减负。
换句话说，它优先优化的是“既然 cache 大概率没了，那就别再背着旧包袱”。

二、Cached microcompact

在理解 `microcompact` 之前，先理解什么是 prompt cache

要看懂 `cached microcompact`，先要知道它到底在保护什么。
它保护的不是“消息文本本身”，而是 **prompt cache 的可复用性**。

这里的 prompt cache，可以先简单理解为：

> 模型服务系统会在服务端，为某些已经计算过的请求前缀保存可复用的计算状态。当前缀保持不变时，后续请求就不需要再把这一大段从头重新计算一遍。

这意味着，在一个长对话里，真正反复发送给模型的内容，并不只是用户这一轮新输入的那几句话，还包括系统拼接出来的大量上下文，例如：

- system prompt
- 历史对话
- assistant 的 tool use
- user 返回的 tool result

这些内容共同组成了请求前部的一大段前缀。而这段前缀中，很多 token 往往在前几轮已经被服务端计算过，因此具备被 cache 复用的价值。

为什么 `tool result` 会成为问题

问题在于，`tool result` 往往很长，而且会随着会话推进不断累积。

例如一次搜索结果、一次文件读取结果、一次网页抓取结果，都可能带来大量 token。
这些结果在刚产生时是有价值的，但随着后续对话继续进行，较早的一批 tool result 往往已经不再重要，却依然占据上下文空间。

于是系统就会面临一个矛盾：

- 一方面，希望把旧的 tool result 去掉，减轻上下文负担
- 另一方面，又不希望因为“删除动作”本身破坏前缀，导致 prompt cache 失效

这正是 `cached microcompact` 要解决的问题。

为什么不能直接删除旧的 `tool result`

最直接的想法当然是：
既然旧 tool result 没用了，那就直接从消息里删掉。

但问题是，一旦直接删除，后续请求的前缀就变了。

因为 prompt cache 依赖的是“前缀稳定”。
如果原来前缀里包含某个旧的 tool result，而现在这个片段被你直接删掉，那么服务端看到的请求前缀就不再是之前那一段，自然也就更难命中已有 cache。

换句话说，系统真正担心的不是“删掉内容会不会影响语义”，而是：

> 这个删除动作会不会让原本已经很贵、已经算过的前缀缓存失效。

---

cached microcompact 做的到底是什么

因此，cached microcompact 的目标并不是传统意义上的“把消息改短”，而是：

> 在尽量不破坏 prompt cache 前缀可复用性的前提下，让旧工具结果在后续请求里逻辑上消失。

它采用的不是普通的消息改写，而是一种 **cache editing** 思路。

也就是说，它并不急着直接修改本地消息文本，而是通过面向缓存层的编辑机制，把某些旧 tool result 标记为“后续请求中不再参与有效上下文”。这样做的重点不是改写消息本身，而是尽量维持服务端对原有前缀缓存的复用能力。

所以从本质上说，cached microcompact 不是摘要压缩，也不是简单删消息，而是：

一种面向 prompt cache 的增量裁剪策略。

三、Time-based microcompact

如果说 cached microcompact 的核心目标是“尽量保住 prompt cache”，那么 time-based microcompact 的逻辑正好相反：**当系统判断缓存大概率已经过期时，就不再为了缓存命中而保留大量旧工具结果，而是直接在发请求前清理它们。**

和 cached microcompact 不同，time-based microcompact 的出发点不是“尽量保住 prompt cache”，而是：

> 当系统判断服务端缓存大概率已经过期时，就不再为了维持缓存命中而保留大量旧工具结果，而是在发请求前直接清理它们。

它判断的不是“内容老不老”，而是“缓存还值不值得保”

这是 time-based microcompact 最关键的设计点。

它并不会去分析某条 tool result 在语义上是否“过时”，也不是在做摘要层面的压缩判断。
它关注的是另一个更工程化的问题：

> 距离上一次 assistant 消息已经过去这么久，服务端的 prompt cache 还有多大概率能继续命中？

源码注释已经把这个意图写得很明确：如果时间间隔已经超过一个较长阈值，例如默认注释里提到的 60 分钟，那么服务端前缀缓存大概率已经冷掉。这意味着即使什么都不做，下一轮请求也很可能要重新计算整段前缀。

既然前缀大概率本来就要重算，那么这时再为了“保缓存”而硬留着大量旧 tool result，就没有太大意义了。
这也是它与 cached microcompact 的根本区别：

- cached microcompact 认为缓存还值得保，所以尽量不动消息正文

- `time-based microcompact` 认为缓存大概率已经不值钱了，所以接受改写消息内容，直接换取更轻的上下文

`Time-based microcompact`具体实现

它的执行流程大致可以概括为：

1. 找出所有可 compact 的工具调用对应的 `tool_use` id
2. 根据 `keepRecent` 规则，保留最近的若干个 tool result
3. 对更早的 tool result，不删除消息块本身，而是把 `content` 替换成固定占位文本，例如 `[Old tool result content cleared]`
4. 统计节省的 token 数量
5. 写入日志和事件
6. 重置 cached microcompact 相关状态
7. 告诉 cache break detection：接下来命中率下降是预期内行为，而不是异常

这里最值得注意的是第三步：
它不是粗暴地把整条消息删掉，而是**保留消息结构，只清空最贵的正文内容。**

这背后其实是一个很稳健的工程取舍。

对于 tool 调用链路来说，`tool_use` 和 `tool_result` 之间通常存在明确的结构关系。
如果直接删除整块消息，虽然 token 降得更多，但也更容易破坏消息配对关系、上下文可读性，甚至触发 API 层的结构约束问题。相反，如果只把 `tool_result.content` 替换成占位文本，那么：

- 消息还在
- 结构还在
- tool_use / tool_result 的对应关系还在
- 但最占 token 的那部分正文已经被拿掉了

所以这不是“删得最狠”的策略，而是“在结构安全前提下删得足够多”的策略。

为什么它至少会保留最近的一部分结果

另一个很有意思的设计是：它不会把所有 tool result 一口气全部清空，而是会通过 `keepRecent` 保留最近若干个，并且给这个值设置下界。

这样做有两个原因。

第一，这是一个工程层面的边界保护。
类似 `slice(-0)` 这类写法很容易造成“本来想一个不留，结果反而全部保留”的陷阱，因此显式设置最小保留值可以减少这类边界错误。

第二，这是一个模型可用性的考虑。
如果把所有工具结果都完全清空，模型就可能失去最近执行轨迹的上下文痕迹，导致后续理解出现断层。适当保留最近的一小部分结果，既能减轻负担，又不至于让模型完全失去近端工具调用线索。

因此，这里的目标并不是“清得越干净越好”，而是：

> 在缓存已经不值得保留的前提下，尽可能移除旧负担，同时保留最基本的近期执行连续性。